Service discovery and Spring Cloud Kubernetes

If you need to migrate existing applications that use one of the client-side service discovery options I mentioned in the previous section, you can use Spring Cloud Kubernetes to make the transition smoother. You can keep your existing service discovery and load-balancing logic in your application. However, instead of solutions like Spring Cloud Netflix Eureka, you can use the *Spring Cloud Kubernetes Discovery Server* for service registry. This can be a convenient way to migrate applications to Kubernetes without changing too much in your application code. For more information, refer to the project documentation: https://spring.io/projects/spring-cloud-kubernetes.

Unless what you're doing requires specific handling of service instances and load balancing in your applications, my recommendation is to migrate over time to using the native service discovery functionality offered by Kubernetes, aiming at removing infrastructural concerns from your applications.

With this general understanding of how service discovery and load balancing are implemented in Kubernetes, let's look at how we can define a Service to expose a Spring Boot application.

7.3.4 Exposing Spring Boot applications with Kubernetes Services

As you learned in the previous section, Kubernetes Services let you expose a set of Pods via an interface that other applications can call without knowing the details about the individual Pod instances. This model provides applications with transparent service-discovery and load-balancing functionality.